

Swap, Don't Route

A 33-vertex counterexample to the undirected network coding conjecture

Written by Opus 5.5

Draft, October 7, 2026

Abstract

We describe a counterexample to the undirected network coding conjecture of Li and Li: a 33-vertex network on which an XOR code achieves rate 1 while routing achieves at most 149/150. The network consists of six cells in a 3×2 grid. In each cell, two messages travel along short paths toward each other's sinks, and the three-step XOR swap exchanges the contents of the two paths. The first step is paid for once per column and the second once per row, each with a single parity bit, and the side information they need rides on extra messages whose shortest paths cross the same links anyway. The idea comes from the bit network of the preprint *Integer multiplication below $n \log n$* [OAI26]. The same network also refutes a published theorem asserting that coding cannot lower the transmission cost in layered networks with uniform edge costs [XLWH14].

1 Introduction

Consider an undirected network with integer edge capacities and a list of unicast sessions. In each session, a source must deliver independent message bits to a sink. *Routing* treats bits as commodities: each bit travels along a path, and the paths share the capacity. The best common rate achievable this way is the maximum concurrent multicommodity flow, MCF. *Network coding* also lets intermediate vertices transmit functions of what they have received, such as XORs. We write NC for the best rate achievable with coding. Routing is a special case of coding, so $\text{NC} \geq \text{MCF}$.

In directed networks coding can beat routing by a wide margin, and the butterfly network is the standard example. In undirected networks the butterfly's advantage disappears, and Li and Li conjectured that coding never helps there [LL04]; see also [HKL06]. The conjecture is also known as the multiple-unicast conjecture.

Conjecture (undirected network coding conjecture). *For every undirected network with unicast sessions, $\text{NC} = \text{MCF}$.*

The conjecture has been proved for some restricted classes of networks (Section 5), and a recent preprint proves it for networks with at most five terminal locations [LQLL26]. It has also been used as a hypothesis for lower bounds. Assuming it, Afshani, Freksen, Kamma and Larsen proved that constant-degree Boolean circuits multiplying two n -bit integers need size $\Omega(n \log n)$ [AFKL19], and Farhadi, Hajiaghayi, Larsen and Shi proved lower bounds for sorting in external memory [FHLS19].

Theorem 1. *There is an undirected network $\mathcal{N}$ with 33 vertices, 91 edges and total capacity 149, together with 30 unit sessions on 7 distinct source–sink pairs, such that*

$$\text{NC}(\mathcal{N}) = 1 \quad \text{and} \quad \text{MCF}(\mathcal{N}) \leq \frac{149}{150}.$$

Hence the conjecture is false. Braverman, Garg and Schwartzman showed that a single network on which coding beats routing can be amplified into a family of networks on which $\text{NC}/\text{MCF} \geq (\log |V|)^c$ for a constant $c > 0$, where $|V|$ is the number of vertices [BGS17]. Applied to $\mathcal{N}$, their result shows that, in the words of their title, network coding in undirected graphs is very helpful. The lower bounds of [AFKL19, FHLS19] assumed the conjecture; the counterexample refutes that assumption but does not show that the bounds themselves fail.

Deleting the hub O and letting each bit it sends start at its recipient instead gives a 32-vertex network with $\text{MCF} \leq 125/126$, at the cost of 13 distinct source–sink pairs instead of 7 (Appendix C).

Section 5 compares $\mathcal{N}$ with the classes of networks on which the conjecture has been proved (Table 2). It also shows that $\mathcal{N}$ refutes Theorem 6.3 of Xiahou, Li, Wu and Huang [XLWH14], which asserts that in a layered network with uniform edge costs, where each session’s source and sink lie in different layers, no code meets the sessions’ rates more cheaply than routing does.

Origin. The construction comes from a recent preprint that claims an integer multiplication algorithm running in time $O(n(\log n)^{1-\kappa})$, for some constant $\kappa > 0$, on a multitape Turing machine [OAI26]. The counterexample does not rely on the multiplication result being correct. The algorithm’s engine is a fast way to permute data on tape. The paper observes in its Section 1.7 that this procedure “forms XOR combinations of intermediate data, even though its final effect is a permutation.” A permutation is pure data movement, so the procedure uses coding where one would expect routing. Section 2 turns the XOR swap at its core into a network where coding beats routing, and ends by tracing the correspondence.

Model. Edges are undirected, and capacities bound traffic in both directions together; in all other respects we follow the standard model of Braverman, Garg and Schwartzman [BGS17, §2 and Appendix A]. Several sessions may share endpoints, so the 30 unit sessions of Theorem 1 amount to 7 sessions: one from O to Z with demand 24, and six from s_{ab} to t_{ab} with demand 1. Appendix B.2 gives variants in which every session has its own endpoints or every capacity is 1. Our code delivers one bit per session with a fixed causal schedule of XORs that stays within every capacity, so it achieves rate 1.

Verification. Two scripts in the accompanying repository check the construction exactly, using only Python’s standard library. Gaussian elimination over $\mathbb{F}_2$ confirms that every sender can compute each bit it sends and that every sink can decode, and the scripts also check every capacity, the layer structure and all hop distances. The script `scripts/verify-network-coding-33.py` checks $\mathcal{N}$, and `scripts/verify-network-coding-32.py` checks the 32-vertex version.

2 The idea: swap, don’t route

The XOR swap. Over $\mathbb{F}_2$, three XOR updates exchange two values:

$$(x, y) \mapsto (x, x \oplus y) \mapsto (y, x \oplus y) \mapsto (y, x).$$

Read this as a network. One path carries x and another carries y , and each update sends one bit from one path to the other. After the three updates the first path holds y and the second holds x . Neither bit is routed along the other path; the paths only trade contents.

Why this can beat routing. Suppose that x must go from s to t and y from O to Z , but the short paths in the network join s to Z and O to t . Routing must still send each bit to its own sink. Coding can send x toward Z and y toward t along the short paths and then swap the contents with three XOR updates between the two paths. This wins if the updates cost less than routing's detours.

One cell. Our network consists of six *cells* arranged in a 3×2 grid, with three rows and two columns, and each cell has its own pair of messages x and y . Figure 1 shows one cell, with subscripts dropped. The vertices lie in six layers, and every edge joins two consecutive layers. Call a path *monotone* if each of its edges goes up one layer. The t -lane $O \rightarrow H \rightarrow p \rightarrow R \rightarrow t$ and the Z -lane $s \rightarrow C \rightarrow q \rightarrow G \rightarrow Z$, named for where they end, are monotone paths of 4 hops each. No path from s to t is monotone (Lemma 3), so routing x needs a detour, 5 hops to climb 3 layers, while y needs 5 hops to climb 5 layers. Counting one *bit-hop* per bit per edge, routing a cell costs 10 while the lanes cost 8, so the swap can save up to 2 bit-hops per cell.

Only s , p , q and t belong to the cell, and Figure 1 shades the other vertices. The *column parity vertex* C is shared by the three cells of its column, the *row parity vertex* R by the two cells of its row, and the hubs O , H , G and Z by all six cells. This sharing is what makes the swap affordable.

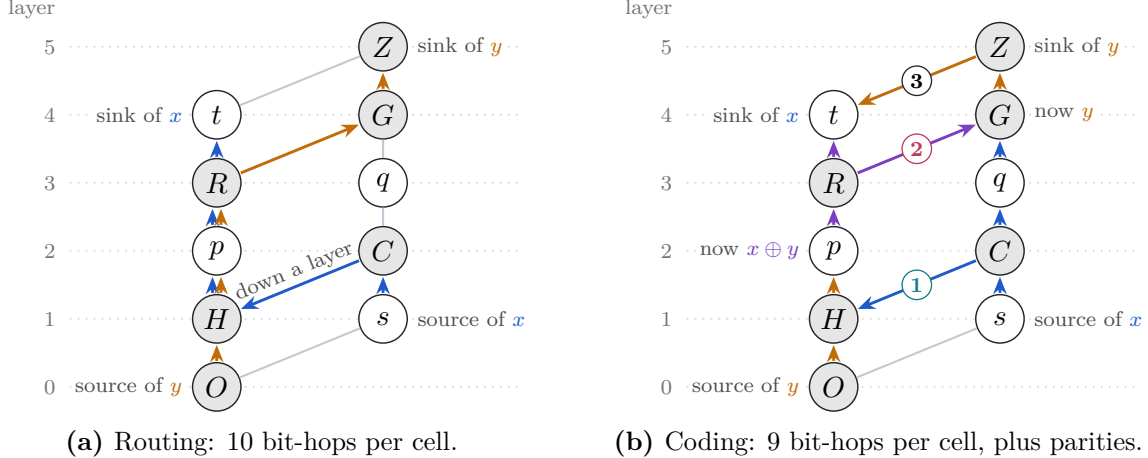


Figure 1: One cell (a, b) of $\mathcal{N}$ with its subscripts dropped. Shaded vertices are shared with other cells. Link colors show what each link carries: blue for x , orange for y and violet for $x \oplus y$. Gray links belong to the network but are not used for x or y in that panel. (a) No path from s to t is monotone, so x needs 5 hops to climb 3 layers. (b) x rides the Z -lane and y the t -lane, and three XOR steps swap their contents. Step 1 costs one bit per column and step 2 one bit per row; only step 3 is paid for per cell. In every figure, teal marks step 1 and rose marks step 2.

Paying for the swap. Each XOR update needs a bit to cross between the lanes. If every cell paid for its own three crossings, coding would cost $8 + 3 = 11$ bit-hops per cell and lose. We

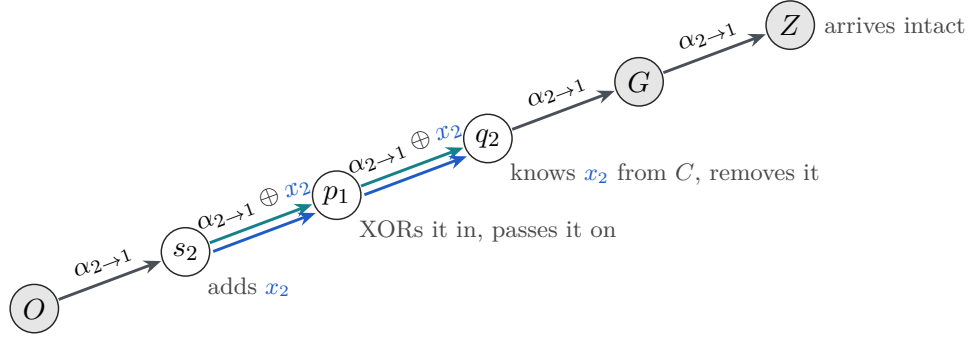


Figure 2: The column carrier $\alpha_{2 \rightarrow 1}$, with the column index dropped. It follows a monotone path from O to Z , five hops in all, which is exactly its distance. On its two column links (teal, as in Figure 3) it also carries x_2 . The vertex p_1 uses this bit in its own XOR and forwards it unchanged. The vertex q_2 already knows x_2 because C sent it there, so it removes x_2 , and the carrier arrives intact.

call the three updates steps 1, 2 and 3. The construction pays for step 3 directly, with one hop $Z \rightarrow t$ per cell, and pays for the first two steps in bulk:

- Step 1 ($y \oplus = x$) is done for a whole column at once. One parity bit per column crosses from the Z -lane to the t -lane, along $C \rightarrow H$.
- Step 2 ($x \oplus = y$) is done for a whole row at once. One parity bit per row crosses back, along $R \rightarrow G$.

Each cell then recovers its own term from the parity. In step 1, for instance, a cell that knows the other bits of its column can cancel them from the column parity, leaving its own.

Side information at no extra cost. The parity trick needs side information: to cancel the other cells' bits, a cell must receive bits that contain them. These bits travel on *cross-cell links*, which join vertices of different cells: *column links*, within a column, serve step 1, and *row links*, within a row, serve step 2. In an undirected network, a link that coding spends on side information is usually one that routing could use to carry the message itself, which is why the butterfly's gain disappears. Here every side-information bit piggybacks on a *carrier*, an extra message from O to Z that crosses that link anyway, on one of its shortest paths (Figure 2): *column carriers* cross column links and *row carriers* cross row links. Routing must pay for the carriers too, so it gains nothing from the cross-cell links. Each carrier travels exactly its distance, so carriers cost the same under coding as under routing.

The count. Per cell, swapping saves the 2-hop detour and spends 1 hop on step 3. Step 1 costs one parity bit per column and step 2 one per row, and carriers cost what they would under routing. So routing needs more bit-hops than coding uses, by

$$\text{cells} - \text{rows} - \text{columns} = 6 - 3 - 2 = 1,$$

which gives 149 bit-hops against 150 (Table 1). This comparison is not a heuristic; it is the routing bound. The capacity of each edge is defined as the number of bits the code sends across it, so the total capacity equals coding's bit-hop count, while routing at rate 1 must spend at least the sum of the sessions' distances (Section 4).

To see what the count gives for other shapes, view the set of cells as a bipartite graph with a vertex for each row and each column and an edge for each cell. When this graph is connected,

the saving cells – rows – columns is one less than its cycle rank, so the count favors coding exactly when the graph has at least two independent cycles. $K_{2,3}$, our 3×2 grid, is the smallest bipartite graph with two. A count of zero or less does not by itself rule out a gap, but for the 2×2 grid an LP solve confirms that there is none. Appendix B.1 gives the count for any grid.

Where the idea comes from. At the core of the procedure of [OAI26] is a fixed linear network, the *bit network* of its Section 3, which swaps two banks of bits with the same three steps $y \oplus = x$, $x \oplus = y$, $y \oplus = x$, but performs each step indirectly: central wires gather sums of many values, each target adds in the sums it needs, and side wires carry copies of neighboring values that cancel the unwanted terms, a cancellation the paper relates to linear index coding [BBJK06]. Its scratch wires may start with arbitrary contents and are restored at the end, which the paper traces to catalytic computation [BCKLS14]. In $\mathcal{N}$ the central wires become the parity bits, the side wires become the cross-cell links, and “restored” becomes “delivered”: a scratch value that must come back intact behaves like a carrier, a message that must arrive intact.

Item	Coding	Routing
x and y of one cell	9	at least 10
One carrier	5	at least 5
One column parity	1	not needed
One row parity	1	not needed
Whole 3×2 grid: 6 cells, 18 carriers, 2 columns, 3 rows	149	at least 150

Table 1: Capacity used by coding and needed by routing, in bit-hops.

3 The network and the code

The six cells (a, b) form a 3×2 grid, with rows indexed by $a \in \{1, 2, 3\}$ and columns by $b \in \{1, 2\}$. We write $\bar{b} = 3 - b$ for the other column. All sums are XORs over $\mathbb{F}_2$.

Vertices. Each cell (a, b) has four vertices s_{ab} , p_{ab} , q_{ab} and t_{ab} . Each column b has a column parity vertex C_b , each row a has a row parity vertex R_a , and there are four hubs O , H , G and Z , for 33 vertices in all. Layer 0 holds O , layer 1 holds H and the s_{ab} , layer 2 the C_b and p_{ab} , layer 3 the R_a and q_{ab} , layer 4 G and the t_{ab} , and layer 5 holds Z .

Messages. There are 30 independent message bits: x_{ab} from s_{ab} to t_{ab} , y_{ab} from O to Z , and carriers from O to Z . For each column b and each ordered pair of distinct rows (j, i) , the column carrier $\alpha_{b, j \rightarrow i}$ will take x_{jb} to p_{ib} in step 1. For each cell (a, b) , the row carrier β_{ab} will take $x_{ab} \oplus y_{ab}$ to $q_{a\bar{b}}$ in step 2.

Edges. The edges of $\mathcal{N}$ are exactly the links the code uses, and the capacity c_e of an edge e is the number of bits the code sends across it, in both directions together. Figure 3 draws the result, which has 91 edges and total capacity 149. Step 1 uses the column links and step 2 the row links. Appendix A lists every message, transmission and edge.

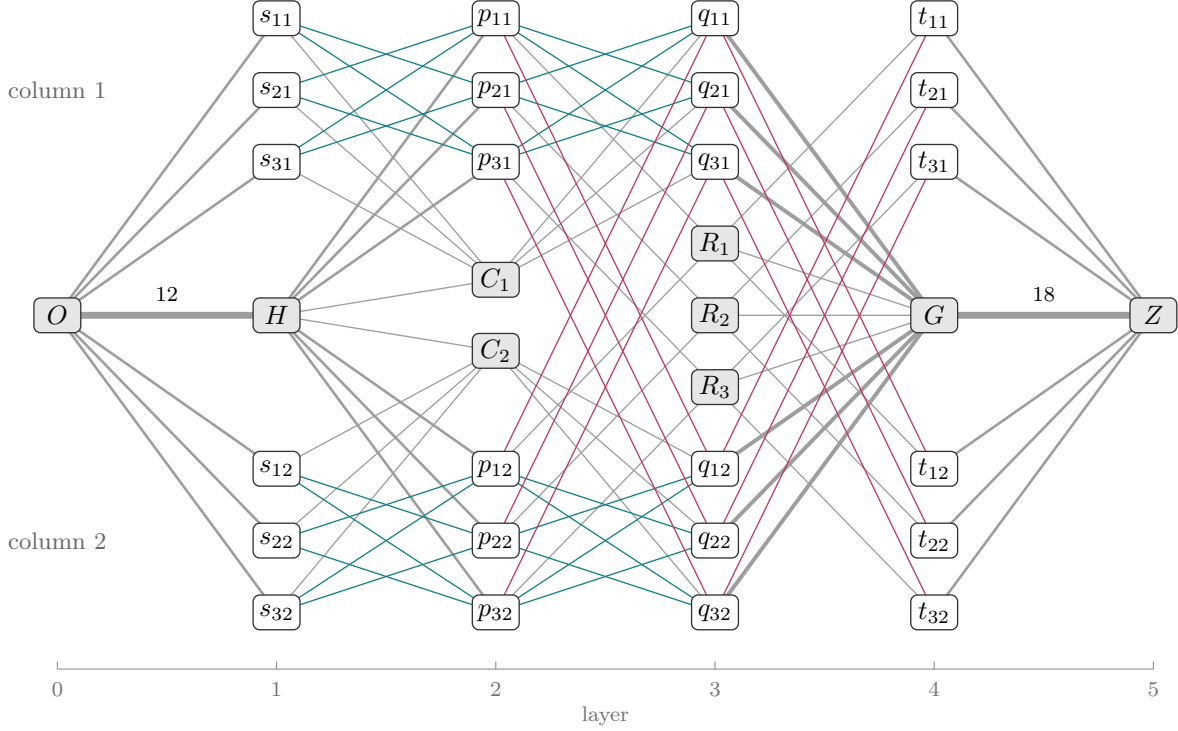


Figure 3: The full network $\mathcal{N}$, drawn by layer from left to right. Column 1 of the grid is drawn above the hubs and column 2 below, with rows 1 to 3 from top to bottom, so the vertices $s_{ab}, p_{ab}, q_{ab}, t_{ab}$ of each cell lie on one horizontal line. Shaded vertices are shared between cells. **Teal:** column links, used in step 1, $s_{jb}p_{ib}$ and $p_{ib}q_{jb}$ ($j \neq i$), which carry $\alpha_{b,j \rightarrow i} \oplus x_{jb}$. **Rose:** row links, used in step 2, $p_{ab}q_{a\bar{b}}$ and $q_{a\bar{b}}t_{ab}$, which carry $\beta_{ab} \oplus x_{ab} \oplus y_{ab}$. **Gray:** every other edge, including the parity links C_bH and R_aG . Line width grows with capacity: the edges Os_{ab} , Hp_{ab} and $t_{ab}Z$ have capacity 2, the edges $q_{ab}G$ have capacity 3, OH and GZ have the capacities shown, and every other edge has capacity 1.

3.1 Step 1: $y \oplus = x$, one column at a time

Fix a column b .

- Each s_{ab} sends x_{ab} up the Z -lane to C_b , which forwards it to q_{ab} and sends the column parity $P_b = x_{1b} \oplus x_{2b} \oplus x_{3b}$ down to H .
- O sends $y_{ab} \oplus \sum_{j \neq a} \alpha_{b,j \rightarrow a}$ to H , which adds P_b and sends the result, u_{ab} , up the t -lane to p_{ab} .
- For each ordered pair of distinct rows (j, i) , O sends the column carrier $\alpha_{b,j \rightarrow i}$ to s_{jb} . The vertex s_{jb} sends $\alpha_{b,j \rightarrow i} \oplus x_{jb}$ along a column link to p_{ib} , which passes it on to q_{jb} .

The vertex p_{ab} XORs what it has received,

$$u_{ab} \oplus \sum_{j \neq a} (\alpha_{b,j \rightarrow a} \oplus x_{jb}) = y_{ab} \oplus P_b \oplus \sum_{j \neq a} x_{jb} = y_{ab} \oplus x_{ab},$$

which we call w_{ab} . The vertex q_{jb} knows x_{jb} from C_b , so it strips x_{jb} from each carrier it receives and sends $\alpha_{b,j \rightarrow i}$ through G to Z .

3.2 Step 2: $x \oplus = y$, one row at a time

Fix a row a . It has two cells, $(a, 1)$ and $(a, 2)$, and after step 1 each p_{ab} holds w_{ab} .

- Each p_{ab} sends w_{ab} up the t -lane to R_a , which forwards it to t_{ab} and sends the row parity $Q_a = w_{a1} \oplus w_{a2}$ up to G .
 - O sends the row carrier β_{ab} through H to p_{ab} , which sends $v_{ab} = \beta_{ab} \oplus w_{ab}$ along a row link to $q_{a\bar{b}}$. That vertex passes v_{ab} on to t_{ab} , which knows w_{ab} from R_a , removes it, and sends β_{ab} to Z .
 - Each q_{ab} knows x_{ab} from step 1 and has received $v_{a\bar{b}}$, so it sends $F_{ab} = x_{ab} \oplus v_{a\bar{b}}$ up to G .
- The hub G adds the row parity and sends

$$F_{ab} \oplus Q_a = x_{ab} \oplus \beta_{a\bar{b}} \oplus w_{a\bar{b}} \oplus w_{a1} \oplus w_{a2} = x_{ab} \oplus w_{ab} \oplus \beta_{a\bar{b}} = y_{ab} \oplus \beta_{a\bar{b}}$$

to Z . The hub Z has received $\beta_{a\bar{b}}$ from $t_{a\bar{b}}$, so it recovers y_{ab} .

3.3 Step 3: $y \oplus = x$, one hop per cell

Z sends y_{ab} down to t_{ab} , which holds w_{ab} from step 2 and computes $x_{ab} = w_{ab} \oplus y_{ab}$.

Proposition 2. *The code of Sections 3.1–3.3, listed in full in Table 3, is causal and zero-error, has block length 1, delivers every message bit to its sink, and sends exactly c_e bits across each edge e . Hence $\text{NC}(\mathcal{N}) = 1$.*

Proof. Table 3 lists the transmissions of the three steps in a causal order: each sender already knows every bit it sends. Sections 3.1–3.3 show the computations, and Table 5 traces them hop by hop for one cell. At the end Z holds every y , α and β bit, and each t_{ab} holds x_{ab} . The capacities were defined as the code’s traffic, so every edge carries exactly its capacity. Hence $\text{NC}(\mathcal{N}) \geq 1$.

Conversely, the edges at O are OH , of capacity 12, and the six edges Os_{ab} , of capacity 2 each, for a total of 24. The 24 sessions from O to Z must all cross this cut, so by the cut bound, which holds for coding as well as routing, $\text{NC}(\mathcal{N}) \leq 1$. $\square$

4 Routing needs capacity 150

Recall that a path is monotone if each of its edges goes up one layer. Every edge joins two consecutive layers, so a path that goes down a layer on d of its edges is $2d$ edges longer than the difference between the layers of its endpoints.

Lemma 3. *For every cell (a, b) , no path from s_{ab} to t_{ab} is monotone. Hence every such path has at least 5 edges.*

Proof. A monotone path from s_{ab} in layer 1 to t_{ab} in layer 4 has exactly three edges, so it consists of s_{ab} , a vertex in layer 2, a vertex in layer 3, and t_{ab} . The layer-2 vertex is a neighbor of s_{ab} , so it is C_b or p_{ib} with $i \neq a$, and the layer-3 vertex is a neighbor of t_{ab} , so it is R_a or $q_{a\bar{b}}$. Among the vertices of layer 3, however, C_b is adjacent only to the q_{kb} , and p_{ib} only to R_i , to the q_{kb} with $k \neq i$, and to $q_{i\bar{b}}$. None of these is R_a or $q_{a\bar{b}}$, because $i \neq a$ and $b \neq \bar{b}$. So no path from s_{ab} to t_{ab} is monotone, and every such path has at least $3 + 2 = 5$ edges. $\square$

Proposition 4. $\text{MCF}(\mathcal{N}) \leq 149/150$.

Proof. Every session needs paths of at least 5 edges: the 24 sessions from O to Z span five layers, and each session from s_{ab} to t_{ab} needs 5 edges by Lemma 3. Take a routing at rate λ , decompose

each session’s flow into paths, and discard cycles. Each of the 30 sessions sends λ units along paths with at least 5 edges, and a unit of flow on a path with h edges uses h units of capacity. Summing over the sessions, the total flow over all edges, counted in both directions, is at least 150λ . It is at most the total capacity, 149. Hence $\lambda \leq 149/150$.¹ $\square$

Theorem 1 follows from Propositions 2 and 4.

5 Comparison with known results

The conjecture has been proved for several classes of networks. Some of these classes are *cut-sufficient*: on them, routing achieves the sparsest-cut bound, which bounds coding as well. The others rest on information inequalities that go beyond cuts. Table 2 lists the classes, the condition that each imposes, and where $\mathcal{N}$ stands. The 32-vertex network $\mathcal{N}'$ of Appendix C falls outside the same classes for the same reasons.

Proved by	Condition	$\mathcal{N}$
Hu [Hu63]	at most 2 source–sink pairs	7 pairs
Liu et al. [LQLL26, Thm. 4]	at most 5 terminal locations	14
Yin et al. [YLLW18]; [LQLL26, Thm. 7]	at most 6 coding vertices	all 33 code
Papernov [Pap76], Lomonosov [Lom85]; [LQLL26, Thm. 6]	demand graph inside K_4 , a 5-cycle, two stars, or a triangle and a star	7 disjoint edges
Okamura and Seymour [OS81], Okamura [Oka83]; [LQLL26, Thm. 5]	planar, each session on one of at most 3 faces	nonplanar (K_5 minor)
Seymour [Sey81]	supply and demand graphs together have no K_5 minor	K_5 minor in the supply graph
Chakrabarti et al. [CFW12]	series-parallel, with no odd spindle	not series-parallel (K_5 minor)
Harvey et al. [HKL06]; Yin et al. [YLLW18]; [LQLL26, Lemma 1]	bipartite, unit lengths, at most one pair at distance above 2	7 pairs at distance 5

Table 2: Classes of networks on which the conjecture is proved, and where $\mathcal{N}$ stands. The results of [Hu63, Pap76, Lom85, OS81, Oka83, Sey81, CFW12] show cut-sufficiency; the others use information inequalities beyond cuts.

The entries can be checked as follows. A *terminal location* is a vertex that is the source or the sink of some session [LQLL26]. $\mathcal{N}$ has seven of each: the sources are O and the s_{ab} , and the sinks are Z and the t_{ab} . A *coding vertex* may combine or decode bits, while the other vertices only forward them [LQLL26]; in our code every vertex sends a combination of bits, decodes a message, or both. The *demand graph* has an edge joining the two ends of each source–sink pair.

¹A floating-point LP solve of the maximum concurrent flow, using HiGHS, gives $0.99333\dots$, equal to $149/150$ up to machine precision, so the bound appears to be tight.

For $\mathcal{N}$ it consists of the seven disjoint edges OZ and $s_{ab}t_{ab}$. Every demand graph allowed by Papernov and Lomonosov has at most two disjoint edges.

The five vertex sets

$$\{G\}, \quad \{H, p_{21}, q_{31}, t_{32}\}, \quad \{p_{22}, q_{21}\}, \quad \{C_2, q_{32}\}, \quad \{p_{31}, R_3\}$$

are connected and pairwise adjacent in $\mathcal{N}$, so its supply graph has a K_5 minor. Hence $\mathcal{N}$ is not planar, and since series-parallel graphs have no K_4 minor, it is not series-parallel either. These sets avoid O , so the same holds for $\mathcal{N}'$.

The last row is the closest. Harvey, Kleinberg and Lehman [HKL06] prove the conjecture for bipartite networks in which every source is at distance 2 from its sink and length 1 on every edge is an optimal dual solution of the routing linear program. Yin et al. [YLLW18, Corollary 1] show that on a connected bipartite graph with unit edge lengths, if at most one ordered source–sink pair is at distance greater than 2, then every code satisfies

$$\sum_e f(e) \geq \sum_i r_i d(s_i, t_i),$$

where $f(e)$ is the traffic across edge e in both directions together and r_i is the rate of session i ; Liu et al. [LQLL26, Lemma 1] allow both orientations of one such pair. This inequality is exactly what our codes violate, with $149 < 150$ on $\mathcal{N}$. Our routing bounds also use length 1 on every edge, but $\mathcal{N}$ has seven ordered pairs at distance greater than 2.

5.1 A refuted theorem on layered networks

Xiahou, Li, Wu and Huang [XLWH14] study the cost version of the conjecture, which is equivalent to the original by linear programming duality [XLWH14, Theorem 4.1]. Each edge e now has a cost, or length, $\ell(e) > 0$ instead of a capacity, and a code that sends $f(e)$ bits across e per time step, in both directions together, costs $\sum_e \ell(e)f(e)$; with unit costs, this is the number of bit-hops of Section 2. Routing every session along shortest paths costs $\sum_i r_i d_\ell(s_i, t_i)$, where d_ℓ is the shortest-path distance under ℓ , and the conjecture says that no code costs less. A *layered network* is one whose vertices are partitioned into layers $V_1, \dots, V_{L+1}$ so that every edge joins two consecutive layers. Theorem 6.3 of [XLWH14] states that for k unicast sessions in a layered network in which each s_i and t_i lie in different layers, the cost advantage of coding over routing is at most $\rho = \max_l \rho_l$, where ρ_l is the ratio of the largest to the smallest cost of an edge between layers l and $l+1$. With unit costs $\rho = 1$, so the theorem says that no code is cheaper than routing.

Proposition 5. *Theorem 6.3 of [XLWH14] is false: with unit edge costs, $\mathcal{N}$ violates it.*

Proof. The network $\mathcal{N}$ is layered, and every session starts and ends in different layers. With unit costs, the code of Proposition 2 delivers every session of $\mathcal{N}$ at rate 1 and costs $\sum_e c_e = 149$, while every routing at rate 1 costs at least 150, as in the proof of Proposition 4. So coding is cheaper by the factor $150/149 > 1 = \rho$. $\square$

The error is in Step 2 of the proof in [XLWH14]. It places the vertices of layer l on a vertical line $x = x_l$ in the plane with the ℓ_∞ norm, with $x_{l+1} - x_l$ equal to the cheapest edge cost between the two layers, and asserts that $d_\ell(s_i, t_i)/\rho \leq \|s_i - t_i\|_\infty$ for every session. With unit costs, $\|s_i - t_i\|_\infty$ is the number of layers between s_i and t_i , so the assertion needs a monotone

shortest path from s_i to t_i . Lemma 3 says that $\mathcal{N}$ has no such path: s_{ab} and t_{ab} are three layers apart but at distance 5.

With this missing hypothesis the theorem does hold: if each s_i is joined to t_i by a monotone path, the cuts between consecutive layers alone give the bound. $\mathcal{N}$ lies just outside this class: its 24 sessions from O to Z have monotone paths, but its six sessions from s_{ab} to t_{ab} do not.

References

- [AFKL19] P. Afshani, C. B. Freksen, L. Kamma, and K. G. Larsen. Lower bounds for multiplication via network coding. In *ICALP*, 2019. [arXiv:1902.10935](#).
- [BBJK06] Z. Bar-Yossef, Y. Birk, T. S. Jayram, and T. Kol. Index coding with side information. In *FOCS*, pages 197–206, 2006.
- [BCKLS14] H. Buhrman, R. Cleve, M. Koucký, B. Loff, and F. Speelman. Computing with a full memory: catalytic space. In *STOC*, pages 857–866, 2014.
- [BGS17] M. Braverman, S. Garg, and A. Schwartzman. Network coding in undirected graphs is either very helpful or not helpful at all. In *ITCS*, 2017. [arXiv:1608.06545](#).
- [CFW12] A. Chakrabarti, L. Fleischer, and C. Weibel. When the cut condition is enough: a complete characterization for multiflow problems in series-parallel networks. In *STOC*, pages 19–26, 2012. [arXiv:1203.4041](#).
- [FHLS19] A. Farhadi, M. Hajiaghayi, K. G. Larsen, and E. Shi. Lower bounds for external memory integer sorting via network coding. In *STOC*, 2019.
- [HKL06] N. J. A. Harvey, R. Kleinberg, and A. R. Lehman. On the capacity of information networks. *IEEE Transactions on Information Theory*, 52(6), 2006.
- [Hu63] T. C. Hu. Multi-commodity network flows. *Operations Research*, 11(3):344–360, 1963.
- [LL04] Z. Li and B. Li. Network coding: the case of multiple unicast sessions. In *Allerton Conference on Communication, Control, and Computing*, 2004.
- [Lom85] M. V. Lomonosov. Combinatorial approaches to multiflow problems. *Discrete Applied Mathematics*, 11(1):1–93, 1985.
- [LQLL26] S. Liu, L. Que, Z. Li, and B. Li. On the multiple-unicast conjecture: beyond cut metrics. Preprint, August 2026. [arXiv:2608.06070](#).
- [OAI26] OpenAI. Integer multiplication below $n \log n$. Preprint, September 23, 2026.
- [Oka83] H. Okamura. Multicommodity flows in graphs. *Discrete Applied Mathematics*, 6:55–62, 1983.
- [OS81] H. Okamura and P. D. Seymour. Multicommodity flows in planar graphs. *Journal of Combinatorial Theory, Series B*, 31(1):75–81, 1981.

- [Pap76] B. A. Papernov. On the existence of multicommodity flows (in Russian). In A. A. Fridman, editor, *Studies in Discrete Optimization*, pages 230–261. Nauka, Moscow, 1976.
- [Sey81] P. D. Seymour. Matroids and multicommodity flows. *European Journal of Combinatorics*, 2(3):257–290, 1981.
- [XLWH14] T. Xiahou, Z. Li, C. Wu, and J. Huang. A geometric perspective to multiple-unicast network coding. *IEEE Transactions on Information Theory*, 60(5):2884–2895, 2014.
- [YLLW18] X. Yin, Z. Li, Y. Liu, and X. Wang. A reduction approach to the multiple-unicast conjecture in network coding. *IEEE Transactions on Information Theory*, 64(6):4530–4539, 2018.

A The complete specification

Table 3 lists all 149 transmissions in a causal order, grouped by step, Table 4 lists the 91 edges, and Table 5 follows the two lanes of one cell. Table 3 uses the abbreviations of Section 3:

$$P_b = x_{1b} \oplus x_{2b} \oplus x_{3b}, \quad u_{ab} = y_{ab} \oplus P_b \oplus \sum_{j \neq a} \alpha_{b,j \rightarrow a}, \quad w_{ab} = x_{ab} \oplus y_{ab},$$

$$Q_a = w_{a1} \oplus w_{a2}, \quad v_{ab} = \beta_{ab} \oplus w_{ab}, \quad F_{ab} = x_{ab} \oplus v_{a\bar{b}}.$$

Step	Bit sent	Route	Count	Hops each
1	$\alpha_{b,j \rightarrow i}$	$O \rightarrow s_{jb}$	12	1
	x_{ab}	$s_{ab} \rightarrow C_b \rightarrow q_{ab}$	6	2
	P_b	$C_b \rightarrow H$	2	1
	$y_{ab} \oplus \sum_{j \neq a} \alpha_{b,j \rightarrow a}$	$O \rightarrow H$	6	1
	u_{ab}	$H \rightarrow p_{ab}$	6	1
	$\alpha_{b,j \rightarrow i} \oplus x_{jb}$	$s_{jb} \rightarrow p_{ib} \rightarrow q_{jb}$	12	2
	$\alpha_{b,j \rightarrow i}$	$q_{jb} \rightarrow G \rightarrow Z$	12	2
2	β_{ab}	$O \rightarrow H \rightarrow p_{ab}$	6	2
	w_{ab}	$p_{ab} \rightarrow R_a \rightarrow t_{ab}$	6	2
	v_{ab}	$p_{ab} \rightarrow q_{a\bar{b}} \rightarrow t_{ab}$	6	2
	β_{ab}	$t_{ab} \rightarrow Z$	6	1
	Q_a	$R_a \rightarrow G$	3	1
	F_{ab}	$q_{ab} \rightarrow G$	6	1
	$F_{ab} \oplus Q_a$	$G \rightarrow Z$	6	1
3	y_{ab}	$Z \rightarrow t_{ab}$	6	1

Table 3: The complete code, executed from top to bottom. A route with several arrows means that each intermediate vertex forwards the bit it received. The code makes 149 single-hop transmissions in total.

Edge	Indices	Cap.	Count	Edge	Indices	Cap.	Count
OH		12	1	$C_b q_{ab}$	each cell	1	6
Os_{ab}	each cell	2	6	$p_{ib} q_{jb}$	rows $i \neq j$	1	12
HC_b	each column	1	2	$p_{ab} R_a$	each cell	1	6
Hp_{ab}	each cell	2	6	$p_{ab} q_{a\bar{b}}$	each cell	1	6
$s_{ab} C_b$	each cell	1	6	$R_a t_{ab}$	each cell	1	6
$s_{jb} p_{ib}$	rows $j \neq i$	1	12	$R_a G$	each row	1	3
$q_{ab} G$	each cell	3	6	$q_{a\bar{b}} t_{ab}$	each cell	1	6
GZ		18	1	$t_{ab} Z$	each cell	2	6

Table 4: The 91 edges of $\mathcal{N}$. Capacities sum to 149.

Hop	t -lane	Z -lane
1	$O \rightarrow H$ $y_{ab} \oplus \sum_{j \neq a} \alpha_{b,j \rightarrow a}$	$s_{ab} \rightarrow C_b$ x_{ab}
2	$H \rightarrow p_{ab}$ $y_{ab} \oplus P_b \oplus \sum_{j \neq a} \alpha_{b,j \rightarrow a}$	$C_b \rightarrow q_{ab}$ x_{ab}
3	$p_{ab} \rightarrow R_a$ $x_{ab} \oplus y_{ab}$	$q_{ab} \rightarrow G$ $x_{ab} \oplus v_{a\bar{b}}$
4	$R_a \rightarrow t_{ab}$ $x_{ab} \oplus y_{ab}$	$G \rightarrow Z$ $y_{ab} \oplus \beta_{a\bar{b}}$
5	$Z \rightarrow t_{ab}$ sends y_{ab} , and t_{ab} computes $x_{ab} = w_{ab} \oplus y_{ab}$	

Table 5: The two lanes of cell (a, b) , the swap of Section 2 written out hop by hop. The t -lane switches from y to $x \oplus y$ at p_{ab} in step 1, the Z -lane switches from x to y at G in step 2, and hop 5 is step 3. Each entry is one transmission, so a cell costs 9 bit-hops.

B Other grids and variants

B.1 Other grids

Table 1 in Section 2 itemizes the count for the 3×2 grid. The construction works for any set of cells in a grid. Step 1 gets one column carrier for each ordered pair of distinct cells in a column, and step 2 one row carrier for each ordered pair of distinct cells in a row; with two columns, there is a single row carrier β_{ab} per cell. Let m be the number of cells, n_r and n_c the numbers of nonempty rows and columns, and ν_b and μ_a the numbers of cells in column b and row a . There are $A = \sum_b \nu_b(\nu_b - 1)$ column carriers and $B = \sum_a \mu_a(\mu_a - 1)$ row carriers. Each cell costs 9 bit-hops, each carrier 5 and each parity bit 1, so

$$\sum_e c_e = 9m + 5(A + B) + n_r + n_c, \quad \text{while routing at rate 1 needs } 5(2m + A + B).$$

The difference is $m - n_r - n_c$. For the full 2×2 grid it is 0, and an LP solve confirms $\text{MCF} = 1$ there. For the 3×3 grid it is 3, with capacity 267 against 270. The checker's `--rows` and `--cols` options reproduce these counts.

B.2 Variants

Separate terminals. Twenty-four of the sessions share the source O and the sink Z . If every unit session must have its own endpoints, attach a fresh source leaf to O and a fresh sink leaf to Z for each of them. This gives 30 unit sessions on 81 vertices, with total capacity 197 against 198 needed.

Unit capacities. Replacing each edge of capacity c by c internally disjoint paths of two edges doubles every distance and the total capacity. This gives a simple graph with unit capacities on 182 vertices and the same bound, $\text{MCF} \leq 298/300 = 149/150$.

C A 32-vertex version

The hub O receives nothing, and the 24 bits it sends are linearly independent. So after a change of coordinates, O can be deleted and each of its bits can start at its recipient instead. This

gives a network $\mathcal{N}'$ with one vertex fewer and a slightly larger gap, at the price of more distinct source–sink pairs.

Theorem 6. *There is an undirected network $\mathcal{N}'$ with 32 vertices, 84 edges and total capacity 125, together with 30 unit sessions on 13 distinct source–sink pairs, such that*

$$\text{NC}(\mathcal{N}') = 1 \quad \text{and} \quad \text{MCF}(\mathcal{N}') \leq \frac{125}{126}.$$

The network. Let $\mathcal{N}'$ be $\mathcal{N}$ with O and its seven edges deleted: OH , of capacity 12, and the six edges Os_{ab} , of capacity 2 each. This removes 24 units of capacity, leaving $91 - 7 = 84$ edges and capacity $149 - 24 = 125$. Every other edge keeps its capacity from Table 4. The layers of $\mathcal{N}$ shift down by one, so H and the s_{ab} now form layer 0 and Z forms layer 4.

The messages. In $\mathcal{N}$, the vertex O sends three kinds of bits in Table 3: $\alpha_{b,j \rightarrow i}$ to s_{jb} , β_{ab} to H , and $y_{ab} \oplus \sum_{j \neq a} \alpha_{b,j \rightarrow a}$ to H . Define

$$z_{ab} = y_{ab} \oplus \sum_{j \neq a} \alpha_{b,j \rightarrow a}.$$

The map $(y, \alpha, \beta) \mapsto (z, \alpha, \beta)$ is invertible over $\mathbb{F}_2$, so the 30 bits x , z , α and β are independent and uniform. Give each bit directly to the vertex that O sent it to:

Bits	Source	Sink	Count
x_{ab}	s_{ab}	t_{ab}	6
z_{ab}	H	Z	6
$\alpha_{b,j \rightarrow i}$, rows $j \neq i$	s_{jb}	Z	12
β_{ab}	H	Z	6

The source–sink pairs are the six pairs (s_{ab}, t_{ab}) , the six pairs (s_{jb}, Z) and the pair (H, Z) , so 13 in all. No vertex starts with shared or dependent information.

Proposition 7. $\text{NC}(\mathcal{N}') = 1$.

Proof. Run the code of Table 3 without its 24 transmissions from O . Each of these transmissions sent a bit that its recipient now holds as a message, so every remaining transmission is computed exactly as before, and the remaining 125 transmissions use every edge of $\mathcal{N}'$ exactly to its capacity. At the end each t_{ab} holds x_{ab} , and Z holds every y_{ab} , $\alpha_{b,j \rightarrow i}$ and β_{ab} , from which it computes each demanded bit $z_{ab} = y_{ab} \oplus \sum_{j \neq a} \alpha_{b,j \rightarrow a}$. Hence $\text{NC}(\mathcal{N}') \geq 1$.

Conversely, each s_{jb} is the source of three sessions, x_{jb} and the two column carriers $\alpha_{b,j \rightarrow i}$, and its three edges $s_{jb}C_b$ and $s_{jb}p_{ib}$ ($i \neq j$) have capacity 1 each. By the cut bound, $\text{NC}(\mathcal{N}') \leq 1$. $\square$

Proposition 8. $\text{MCF}(\mathcal{N}') \leq 125/126$.

Proof. $\mathcal{N}'$ is a subgraph of $\mathcal{N}$, so by Lemma 3 every path from s_{ab} to t_{ab} still has at least 5 edges. The other 24 sessions start in layer 0 and end at Z in layer 4, so they need at least 4 edges. As in Proposition 4, routing at rate λ uses at least $(6 \cdot 5 + 24 \cdot 4)\lambda = 126\lambda$ units of capacity, and only 125 are available. Hence $\lambda \leq 125/126$.² $\square$

Theorem 6 follows from Propositions 7 and 8.

²A floating-point LP solve with HiGHS gives 0.992063..., equal to 125/126 up to machine precision.